CSD 302 Design and Analysis of Algorithms

Quiz 3 Sem II 2019-20, Max 20, Time 40 mins

Note (i) You can quote any result covered in the lectures without proof but any other claim should be formally justified.
(ii) Your description should be in a pseudo-language - not a code. Mention clearly any data structures that you use including how the input and output is stored.

(iii) **This is an online test and you are expected to follow the honor code of not taking help from any sources iincluding internet, books or another person. You must sign off your answer sheet with the following signed declaration -**
"I declare that I have complied with the honor code of the online examination."

1. Given a tree (not necessarily binary) with edges having real valued weights, design an efficient algorithm for APSP and analyze the running time. Is your algorithm optimal ? **(10 marks)**

Answer:

Since this is a tree, an optimal solution would be to find the shortest path between two nodes in terms of number of edges between, Since tree is basically an acyclic graph, the shortest path goes through some common ancestor and any other path would also go through an ancestor, but we are interested in the closest one. I have read about something called **Lowest Common Ancestor (LCA)** and that seems so be a good idea for this problem.

In a nutshell, what this does is that for every paid of vertices, it finds the node which is their LCA. This is done with the help of a kind of preorder traversal, for every root we check the left and right subtrees for the presence of a or b if (a,b) is the pair under consideration. A subtree returns a null value to its parent if neither a nor b are found. Otherwise it returns a or b. If any node receives both non-null values, it is the LCA of (a,b). This node's value is passed up all the way up to the root now, and the it is stored at lca(a,b).

Now distances, from root to each node are calculated.

Shortest path between a and b can now be given by **d(a) + d(b) – 2*d(lca(a,b))**

We see that finding the LCA for a pair of vertices, finding distances and the shortest path computation can all clearly be done in linear time.

Hence, the overall complexity of this algorithm is **O(n).**

Yes, this is OPTIMAL.

2. In the 0-1 Knapsack problem for n elements, suppose we are given the information that the profit of every item is bounded by n^3. How would you design a polynomial time algorithm for this case ? If you use Dynamic Programming, then there should be a relevant recurrence with well-defined parameters and justification, the description of the table and how the entries are being computed. **(10 marks)**

Answer:

We can sort the elements by weight and construct a DP table whose columns start from 0 and go to W, where W is the capacity. Rows from 1 to n. Entry (I,j) is the optimal solution for the same problem but with I elements and j capacity of the knapsack.

At each entry we calculate the optimal value as the max of the optimal value of the item is selected or if it isn't

This means that arr[i,]] = max{arr[i-1,j],arr[i-1,j-wi]}, where wi is the weight of item i.

The base case is the first row, with just one item that is filled with 0 for all j<wi, and wi for all j>=wi.

Since the calculation of each arr[i][j] involves examining at most nitems, and there are at most W values in the row, The complexity of this algorithm is O(nW)

I DECLARE THAT I HAVE COMPLIED WITH THE HONOUR CODE OF THE ONLINE EXAMINATION